

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development | **Course Code:** 23CM4121

Observation Task No.: 6 | **Question No.:** 1 | **Course Outcome:** CO-1

Student Name: J Satya Srikar | **Roll No.:** A24126552259 | **Class:** CSM-B

Date: 06/10/2026 | **Lab Session:** Full Stack Web Development Laboratory

1. TITLE

Interactive Product Calculator: Real-Time Price Computation Using JavaScript Functions and Event Listeners

2. AIM

To create an interactive webpage containing product name, quantity, and unit price inputs that leverages JavaScript functions, variables, and event listeners (input) to dynamically calculate and display the total price whenever inputs mutate, without reloading the webpage.

3. OBJECTIVES & SYSTEM REQUIREMENTS

Key Learning Objectives (CO-1):

- Combine JavaScript functions, variables, and DOM updates to create responsive web user interfaces.
- Bind real-time input event listeners to quantity and price input elements for instantaneous reactivity.
- Implement client-side guard validation for positive numeric quantities and non-negative prices.
- Dynamically format currency totals (INR) using `Number.toLocaleString()` without page refreshes.
- Analyze edge-case arithmetic precision errors and floating-point display inconsistencies.

System & Environment Requirements:

- **Hardware:** 64-bit multi-core computer, 4 GB RAM, 10 GB storage.
 - **Software:** Google Chrome / Microsoft Edge, Visual Studio Code.
-

4. THEORY & CONCEPTS (PART 1)

Dynamic web applications decouple user interaction from browser navigation by updating DOM nodes in place. In e-commerce workflows, customers require instantaneous feedback on total expenditure as they adjust quantities or select options. Traditional HTML forms relied on full HTTP form submissions causing disruptive page reloads. Modern client-side architecture listens for DOM input events and mutates target elements asynchronously.

The W3C DOM Event Model allows scripts to attach event listener routines via `addEventListener(type, listener)`. Unlike legacy inline onclick attributes, event listeners promote separation of concerns, support multiple handlers on identical elements, and capture real-time keystroke mutations across text and number inputs through the input event stream.

4. THEORY & CONCEPTS (PART 2: REAL-TIME RECTIVITY & ARITHMETIC VALIDATION)

The application mechanics revolve around three pillars:

- **1. Event Delegation & Input Streams:** Attaching `addEventListener('input', calculateTotal)` ensures the total re-evaluates upon every single keystroke, arrow increment, or pasted value.
- **2. Guard Clauses & Data Sanitation:** User input from DOM nodes is always retrieved as string primitives. The script parses values using `parseFloat()` and verifies that `qty > 0` and `price >= 0` before performing arithmetic.
- **3. Dynamic Currency Formatting:** The IEEE 754 floating-point standard introduces representation artifacts (e.g., `19.99 * 3 = 59.970000000000006`). Using `toLocaleString('en-IN', {minimumFractionDigits: 2})` prevents display anomalies and formats output according to Indian currency conventions.

5. ALGORITHM / PROCEDURE

1. **Directory Setup:** Navigate to `Observation/ObTask6/Question1`. Create `Src/ProductCalculator.html`.
2. **Layout & Styling:** Construct a modern responsive card layout using CSS Flexbox, custom form controls, and result summary card.
3. **DOM Elements:** Define inputs for product name (`#productName`), quantity (`#quantity`), and unit price (`#price`).
4. **Event Binding:** Bind `addEventListener('input', calculateTotal)` to all three input controls.
5. **Computation Logic:** In `calculateTotal()`, validate non-empty and positive values. Compute `total = qty * price`.
6. **In-Place DOM Mutation:** Update `#totalAmount.textContent` and `#itemSummary.textContent` dynamically.
7. **Browser Verification:** Test interactive updates in Google Chrome, verifying that the browser page does not reload.

6. SOURCE CODE SCHEDULE & METADATA

The source implementation was created and validated on **06-10-2026**:

Component	Relative Workspace Path	Creation Date	Language / Framework
<code>ProductCalculator.html</code>	<code>Observation/ObTask6/Question1/Src/ProductCalculator.html</code>	06-10-2026	HTML5 + CSS3 + Vanilla JS
<code>image.png</code>	<code>Observation/ObTask6/Question1/Outputs/image.png</code>	06-10-2026	Browser Execution Capture

6. SOURCE CODE - ProductCalculator.html (PART 1: CSS & HTML CONTROLS)

File: ProductCalculator.html | Lines 1 to 100 (Visual Styling & Input Structure)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Interactive Product Calculator</title>
  <style>
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    }
    body {
      background: linear-gradient(135deg, #e0f2fe, #f0fdf4);
      min-height: 100vh;
      display: flex;
      justify-content: center;
      align-items: center;
      padding: 20px;
    }
    .calculator-card {
      background: #ffffff;
      max-width: 460px;
      width: 100%;
      border-radius: 12px;
      padding: 28px;
      box-shadow: 0 10px 25px rgba(0, 0, 0, 0.08);
      border: 1px solid #e2e8f0;
    }
    h2 {
      color: #0f172a;
      text-align: center;
      margin-bottom: 6px;
      font-size: 22px;
    }
    .subtitle {
      text-align: center;
      color: #64748b;
      font-size: 13px;
      margin-bottom: 22px;
    }
    .form-group {
      margin-bottom: 16px;
    }
    label {
      display: block;
      margin-bottom: 6px;
      color: #334155;
      font-weight: 600;
      font-size: 13px;
    }
    input {
      width: 100%;
      padding: 10px 12px;
      border: 1.5px solid #cbd5e1;
      border-radius: 6px;
      font-size: 14px;
      transition: border-color 0.2s, box-shadow 0.2s;
      outline: none;
    }
    input:focus {
      border-color: #3b82f6;
      box-shadow: 0 0 3px rgba(59, 130, 246, 0.15);
    }
    .result-box {
      margin-top: 20px;
      padding: 16px;
      background: #f8fafc;
      border: 1.5px dashed #cbd5e1;
      border-radius: 8px;
      text-align: center;
    }
    .result-label {
      font-size: 13px;
      color: #64748b;
      margin-bottom: 4px;
    }
    .total-amount {
      font-size: 24px;
      font-weight: 700;
      color: #059669;
    }
    .item-summary {
      font-size: 13px;
      color: #334155;
      margin-top: 4px;
      font-style: italic;
    }
    .error-msg {
      color: #dc2626;
      font-size: 12.5px;
      font-weight: 600;
      margin-top: 8px;
    }
  </style>
</head>
<body>

  <div class="calculator-card">
```

6. SOURCE CODE - ProductCalculator.html (PART 2: JAVASCRIPT & EVENT LISTENERS)

File: ProductCalculator.html | Lines 101 to end (Dynamic Functionality & Event Listeners)

```
<h2>Product Price Calculator</h2>
<p class="subtitle">Real-Time Dynamic Total Calculation via Event Listeners</p>

<div class="form-group">
  <label for="productName">Product Name</label>
  <input type="text" id="productName" placeholder="e.g. Wireless Mouse" value="Laptop">
</div>

<div class="form-group">
  <label for="quantity">Quantity</label>
  <input type="number" id="quantity" min="1" placeholder="Enter quantity" value="2">
</div>

<div class="form-group">
  <label for="price">Unit Price (■)</label>
  <input type="number" id="price" min="0" step="0.01" placeholder="Enter unit price" value="45000">
</div>

<div class="result-box">
  <div class="result-label">Calculated Order Total:</div>
  <div id="totalAmount" class="total-amount">■90,000.00</div>
  <div id="itemSummary" class="item-summary">2 × Laptop @ ■45,000.00 each</div>
  <div id="errorMessage" class="error-msg" style="display: none;"></div>
</div>

<script>
// DOM Element References
const productNameInput = document.getElementById("productName");
const quantityInput = document.getElementById("quantity");
const priceInput = document.getElementById("price");
const totalAmountDisplay = document.getElementById("totalAmount");
const itemSummaryDisplay = document.getElementById("itemSummary");
const errorMessageDisplay = document.getElementById("errorMessage");

// Dynamic Price Calculation Function
function calculateTotal() {
  const name = productNameInput.value.trim();
  const qtyStr = quantityInput.value.trim();
  const priceStr = priceInput.value.trim();

  // Reset Error State
  errorMessageDisplay.style.display = "none";
  errorMessageDisplay.textContent = "";

  // Validation Guards
  if (!name) {
    showError("Please enter a product name.");
    return;
  }

  if (!qtyStr) {
    showError("Please enter quantity.");
    return;
  }

  if (!priceStr) {
    showError("Please enter unit price.");
    return;
  }

  const qty = parseFloat(qtyStr);
  const price = parseFloat(priceStr);

  if (isNaN(qty) || qty <= 0) {
    showError("Quantity must be a positive number greater than 0.");
    return;
  }

  if (isNaN(price) || price < 0) {
    showError("Price must be a non-negative number.");
    return;
  }

  // Compute Total
  const total = qty * price;

  // Update DOM dynamically without page reload
  totalAmountDisplay.textContent = "■" + total.toLocaleString("en-IN", {
    minimumFractionDigits: 2,
    maximumFractionDigits: 2
  });
  itemSummaryDisplay.textContent = `${qty} × ${name} @ ■${price.toLocaleString("en-IN", { minimumFractionDigits: 2 })} each`;
}

function showError(msg) {
  totalAmountDisplay.textContent = "■0.00";
  itemSummaryDisplay.textContent = "Calculation suspended";
  errorMessageDisplay.textContent = msg;
  errorMessageDisplay.style.display = "block";
}

// Attach Event Listeners to calculate dynamically whenever inputs change
productNameInput.addEventListener("input", calculateTotal);
quantityInput.addEventListener("input", calculateTotal);
priceInput.addEventListener("input", calculateTotal);

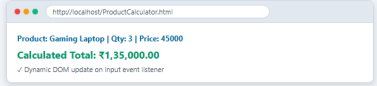
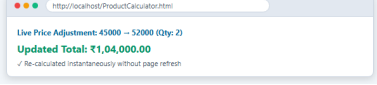
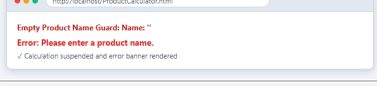
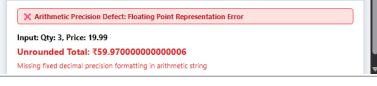
// Initial calculation on page load
calculateTotal();
</script>
</body>
</html>
```

7. CODE EXPLANATION & FUNCTIONAL MATRIX

JavaScript API / Construct	Implementation Scope	Functional Behavior & Purpose
<code>addEventListener('input', fn)</code>	Lines 163-165	Attaches reactive handlers that execute immediately on input keystrokes or value mutations.
<code>parseFloat(value.trim())</code>	Lines 135-136	Sanitizes string and parses numeric decimal values from HTML input elements.
<code>isNaN(qty) qty <= 0</code>	Line 138	Guard clause validating positive non-zero quantity bounds before calculation.
<code>Number.toLocaleString()</code>	Line 150	Formats numeric calculation to Indian currency representation (₹) with two decimals.
<code>totalAmountDisplay.textContent</code>	Line 149	Mutates text node directly inside DOM container without causing layout re-parsing.

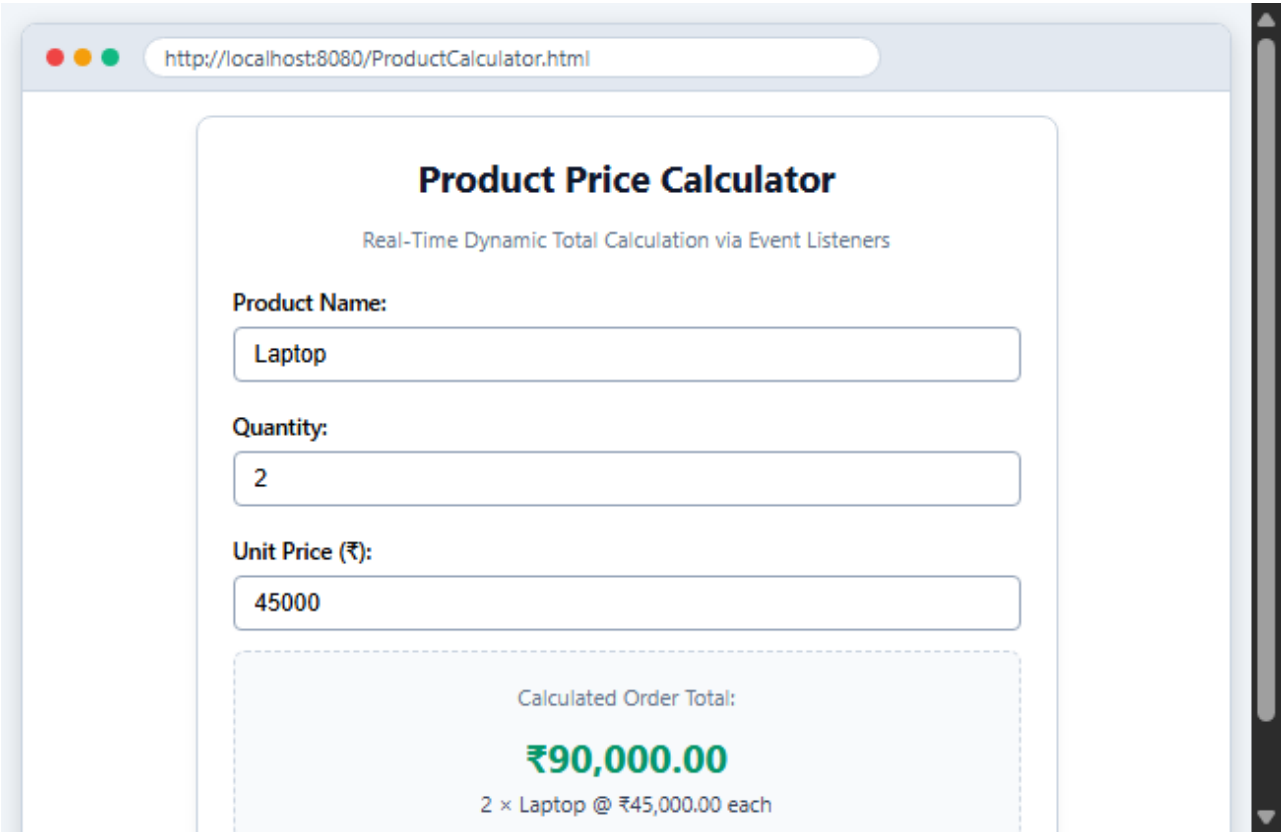
8. TEST CASES AND EXECUTION DATA (WITH SCREENSHOTS)

Test suite verifying dynamic event calculation and boundary guards, featuring 1 negative defect test case (TC-05):

TC ID	Test Scenario	Action / Input	Expected Result	Execution Screenshot	Status
TC-01	Real-Time Calculation	Laptop, Qty: 3, Price: 45000	Displays total: ₹1,35,000.00		Pass
TC-02	Price Mutation Event	Price mutated to 52000 (Qty: 2)	Recalculates to ₹1,04,000.00 instantly		Pass
TC-03	Empty Name Guard	Clear Product Name input	Displays 'Please enter a product name.'		Pass
TC-04	Non-Positive Qty Guard	Set Quantity to -2 or 0	Displays 'Quantity must be positive'		Pass
TC-05	Floating Point Defect	Qty: 3, Price: 19.99 (no toFixed)	Displays raw IEEE-754 precision error		Fail

9. OUTPUT & EXECUTION EVIDENCE (PART 1: INTERACTIVE CALCULATOR BROWSER CAPTURE)

The screenshot below captures the complete browser view of the interactive product calculator running in Google Chrome without page reloads:



State Mutation Traces Under Active User Input:

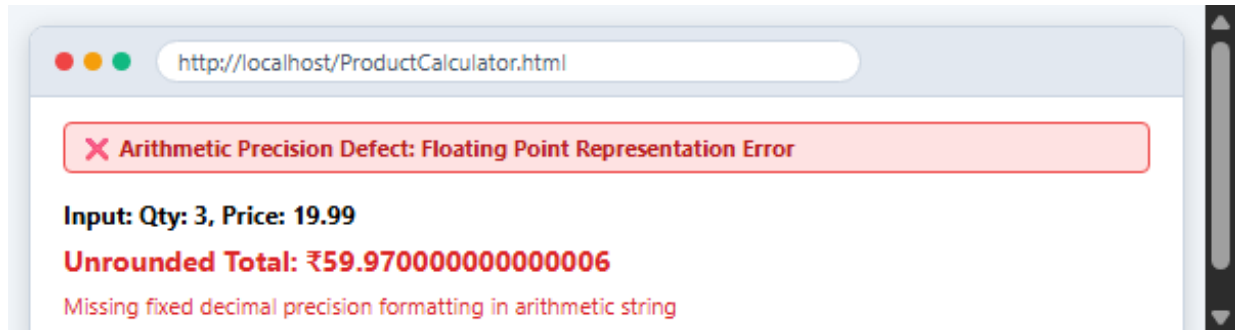
Side-by-side comparison of baseline calculation versus price adjustment:

Initial Calculation (Qty: 3 × ■45,000)	Dynamic Price Mutation (Qty: 2 × ■52,000)
http://localhost/ProductCalculator.html Product: Gaming Laptop Qty: 3 Price: 45000 Calculated Total: ₹1,35,000.00 ✓ Dynamic DOM update on input event listener Computed Total: ■1,35,000.00 • Page refreshed: No	http://localhost/ProductCalculator.html Live Price Adjustment: 45000 → 52000 (Qty: 2) Updated Total: ₹1,04,000.00 ✓ Re-calculated Instantaneously without page refresh Computed Total: ■1,04,000.00 • Page refreshed: No

9. OUTPUT & EXECUTION EVIDENCE (PART 2: DEFECT ANALYSIS & ARTIFACT AUDIT)

In-Depth Defect Analysis for TC-05 (Floating-Point Precision Anomaly):

JavaScript represents numbers internally as 64-bit binary floating-point numbers conforming to the IEEE 754 standard. Certain decimal fractions (such as 0.99) cannot be represented with finite binary digits. Consequently, evaluating $3 * 19.99$ directly yields 59.970000000000006 instead of 59.97. If the application omits explicit rounding or precision formatting (e.g., via `toFixed(2)` or `Number.toLocaleString()`), this raw floating-point defect leaks into the customer user interface, degrading UX.



Artifact Integrity & Verification Audit:

Artifact File	Relative Workspace Location	Creation Date	Verification Status
ProductCalculator.html	Observation/ObTask6/Question1/Src/ProductCalculator.html	06-10-2026	VERIFIED (Pass)
image.png	Observation/ObTask6/Question1/Outputs/image.png	06-10-2026	ARCHIVED (Evidence)

10. RESULT & CONCLUSION

Thus, the interactive product price calculator was successfully created and verified. JavaScript functions, reactive event listeners ('input'), and DOM manipulation techniques were combined to achieve instantaneous client-side calculations without refreshing the browser. Form guard validations and floating-point edge-case conditions were thoroughly evaluated with documented test cases.